

Executive Summary

We propose **TypeScore**, an AI-powered web tool that quantifies a brand's typography quality on a 0–100 scale. Inspired by NPS/SEO scores, TypeScore audits any public website (and linked documents) to measure **Brand Consistency**, **License Compliance**, **Web Performance**, **Accessibility**, and **Developer Experience**. It crawls a site with Playwright (headless Chromium), extracts CSS fonts, and runs Lighthouse (for performance) and axe-core (for accessibility) on up to 5 pages. It also parses PDFs/Office files (via pdfjs-dist and Office parsers) to find embedded fonts. Results include an overall score and breakdown (with detailed violations), plus AI-powered suggestions (e.g. licensed Monotype alternatives for unauthorized fonts). A free demo generates leads for Monotype Connect; gated full reports feed HubSpot. In Phase 2 we'll add email gating, SharePoint integration, industry benchmarking, and Phase 3 brings continuous monitoring, white-labeling, and subscriptions. This report details the product vision, personas (brand teams, legal, developers, agencies, accessibility officers, sales), business model, technical architecture (crawler, queue, DB schema), scoring algorithms, AI features, security/privacy, scalability/costs, integration with Monotype Connect and HubSpot, competitive context, accessibility best practices, deliverables (wireframes, APIs, JSON sample), and a phased roadmap (with mermaid Gantt).

Product Concept

TypeScore is a **public web tool**: the user enters any URL and within ~60 seconds receives a score (0–100) summarizing how well the site's typography aligns with brand guidelines, license rules, performance, and accessibility. It checks for unauthorized fonts, accessibility violations, Core Web Vitals, and technical font issues. The five weighted metrics are:

- **Brand Consistency (20%)** – measures how many different fonts are used vs. the brand's approved set, plus structural consistency (e.g. heading vs body fonts).
- **License Compliance (30%)** – flags embedded fonts that may violate licensing (e.g. a desktop font used on the web). Unrecognized or unlicensed fonts incur heavy penalties.
- **Web Performance (20%)** – uses Lighthouse to score Core Web Vitals (LCP, CLS, FID) and font loading (e.g. large font files, many requests). Poor performance (large WOFF2 files, excessive preloads) lowers the score.
- **Accessibility (15%)** – runs axe-core checks (the same engine used by Lighthouse [38†L185-L193]). It enforces WCAG contrast ($\geq 4.5:1$) and text sizing, warns on text-in-images, small text, or insufficient line spacing [8†L171-L179] [19†L243-L252] .
- **Developer Experience (15%)** – measures technical debt: e.g. number of font files, unused font variants, CSS font-face issues, use of deprecated units. More fonts or poor practices reduce the score.

By turning typography into a single "TypeScore", brands can **benchmark themselves** and create urgency to fix issues (just as SEO and NPS scores drive action). Any brand with a low TypeScore is a ready lead for Monotype Connect: the report automatically suggests Monotype fonts as replacements and invites the user to "unlock your brand" by licensing fonts via Monotype. Agencies can use TypeScore as a client-reporting

tool (even white-labeled). The immediate business value is *top-of-funnel demand generation*: every low score generates evidence-backed leads for Monotype Connect sales.

User Personas

- **Brand/Marketing Teams:** Care about consistent brand identity. They want all touchpoints (web, print, docs) using approved typefaces. TypeScore helps them enforce brand style guides and detect stray “rogue” fonts.
- **Legal/Compliance Officers:** Worried about font licensing risks. They need to know if the site or documents use fonts outside licensed scope. TypeScore flags unauthorized or legacy fonts (avoiding retroactive fees or legal action [\[31†L109-L117\]](#)).
- **Web Developers:** Focused on performance and maintainability. They benefit from alerts about heavy font loads, missing subsets, or outdated practices (e.g. using only uppercase all caps). TypeScore suggests optimizations (e.g. using WOFF2, fewer font weights).
- **Accessibility Officers:** Ensures sites meet WCAG and Section 508 standards. They use TypeScore to catch issues like low contrast, small text, or poor text spacing. The tool enforces best practices (e.g. relative font sizes for zoom [\[19†L264-L272\]](#)).
- **Agencies/Consultants:** Use TypeScore to audit client sites and show quantifiable improvement. They appreciate a white-label version for client reports, industry benchmarking, and ongoing monitoring subscriptions.
- **Sales & Exec:** View TypeScore as a lead-generation and upsell engine. Every low score identifies a pain point and presents a clear path to fix it with Monotype Connect. It provides sales teams with evidence (“14 unauthorized fonts found”) to justify license purchases.

Business Model & Go-to-Market

- **Lead Generation (Free Tool):** TypeScore is offered free for up to 5 pages. Users get an instant score and summary. Any low score triggers a call-to-action (CTA) to Monotype Connect (“Fix your TypeScore by licensing fonts”). This drives inbound leads.
- **Gated Reports & HubSpot Integration:** Full detailed reports (including all violations and recommendations) are behind an email gate. Collected emails are fed into HubSpot, creating warm leads for sales. We also track corporate domains to prioritize Enterprise outreach.
- **Continuous Monitoring Subscription (Phase 3):** For a monthly fee, brand teams can subscribe to re-scan their domain(s) daily/weekly, track trends, and enforce compliance. Alerts notify them of any new unauthorized font usage.
- **White-Label API (Phase 3):** Agencies or large organizations can integrate the TypeScore engine into their own tools or brand it for internal use (via a REST API and white-label license). This creates a new B2B revenue stream.
- **Monotype Connect Synergy:** Every brand with a low score is a qualified lead for Monotype Connect. We leverage the TypeScore findings as “quantified evidence” to pitch Connect. For example, “Your TypeScore is 38 – 14 unlicensed fonts detected” creates urgency. We can integrate Connect’s font database to automatically validate and suggest licensed replacements.
- **Marketing & Outreach:** Use demos (even scanning judges’ or known brand sites live at events), content marketing (“Typography best practices”), and partnerships (e.g. with accessibility consultancies). The tool itself serves as a conversation starter at conferences.
- **KPIs:** Key metrics include number of scans, lead conversion rate (emails collected / scans), demo requests, trial sign-ups for Connect, and eventual license sales attributable to TypeScore leads.

Competitive Landscape & Benchmarking

Currently **no industry-standard “font health” score** exists. There are general accessibility tools (Lighthouse, WAVE) and font managers (Monotype Connect itself) but no dedicated brand typography auditor. TypeScore would be **first to market**, akin to how Lighthouse scores led SEO tools.

We would build an **internal benchmark dataset** by scanning a diverse set of brand sites (e.g. Fortune 500, tech, retail, finance, etc.) to establish average scores. For example, we might find that financial services sites average 80/100 in Brand Consistency but only 60 in Accessibility. These benchmarks could be shown to new users: “Your Brand Consistency is 65, vs. average 78 in your industry.” (Methodology: crawl e.g. Alexa/top sites per industry).

Additionally, we’ll monitor emerging competitors: some digital agencies offer manual typography audits, but nothing automated with AI. Open-source tools (e.g. Google’s Chrome DevTools Accessibility audit) exist but focus on WCAG rather than brand metrics. Thus TypeScore occupies a unique niche.

Technical Design

High-Level Architecture

1. **Frontend:** A web UI built in Next.js + Tailwind (deployed on Vercel). Users input a URL and see results.
2. **Backend/API:** Node.js (or Go/Python) server running on Railway (or AWS/GCP). Exposes a REST API (`POST /scan` , `GET /scan/{id}`) and interfaces with the queue and database.
3. **Job Queue:** BullMQ (Redis-based) handles asynchronous scan jobs. Upon request, a job is enqueued with the target URL and options.
4. **Worker Service:** A separate container picks up jobs and runs the scan:
5. **Crawling:** Use Playwright (headless Chromium) to load the page fully (including JS-rendered content) and extract resources. Crawl up to 5 pages (e.g. homepage and top-level nav links).
6. **CSS & Font Extraction:** For each page, parse all CSS (inline and external) to extract `@font-face` rules and usage. Record font names, URLs, and where they’re used.
7. **Document Parsing:** Inspect page for links to PDFs or Office docs. Fetch those (if public) and use `pdfjs-dist` to parse PDF font references, and an Office file parser (e.g. unzip docx/pptx) to list embedded fonts.
8. **Lighthouse Audits:** Run Lighthouse (via `chrome-launcher` or Playwright plugin) to get Performance (Core Web Vitals) and Accessibility scores. Record metrics like LCP, CLS, TBT, and A11y issues.
9. **axe-core Checks:** Optionally run axe-core independently for additional accessibility detail (though Lighthouse already uses axe `38†L185-L193`).
10. **License Heuristics:** Compare extracted fonts against a known licensed fonts list (Monotype’s database, Google Fonts list, etc.). Flag fonts not on the whitelist of web-safe or licensed fonts. We may fingerprint fonts by names or by downloading and hashing (to detect proprietary fonts disguised via CDN).

11. **Anomaly Detection (AI):** Use simple rules or ML to detect unusual fonts (e.g. an out-of-place decorative font used once) and flag them as anomalies for user review.
12. **Compute Scores:** Based on collected data, calculate each metric's score using defined algorithms (see *Scoring Model*).
13. **Store Results:** Insert findings into PostgreSQL (scan summary and detailed tables).
14. **Database:** PostgreSQL stores scan records, pages, fonts, and issues. A proposed schema is:

Table	Columns (Sample)
scans	id, url, created_at, completed_at, overall_score (int), brand_score, license_score, perf_score, access_score, dev_score
pages	id, scan_id, page_url, lighthouse_perf_score, cwv_lcp, cwv_cls, cwv_fid, num_fonts_on_page
fonts	id, scan_id, page_id (nullable), font_family, source (CSS/embedded/PDF), url, is_system_font, license_status (string)
issues	id, scan_id, type (Brand/License/Perf/Access/Dev), description, element_selector, severity
users (opt.)	id, email, first_name, last_name, company, scans_gated_accessed

1. **Reporting UI:** After scan, show an overview with a gauge/dial for the overall TypeScore and each metric. Below, list specific violations: e.g. "Brand: Found 5 fonts (approved: 2). 3 extraneous fonts used" or "License: 2 fonts likely unlicensed: FontX, FontY". Each issue links to the offending page/element. Provide Monotype CTAs ("Learn more about licensing and align your fonts").
2. **CTA & Lead Capture:** The result page includes a button/link to Monotype Connect. For gated mode, the detailed breakdown (or PDF export) is locked behind an email form. Submitted info is sent to HubSpot via API.

Crawler Details

- **Playwright Setup:** Launch a headless Chromium instance with network interception. Set a crawl depth (e.g. 5 pages) and domain filter (stay on same origin). Extract `<link href>` and `<script>` to find CSS, fonts.
- **Font Extraction:** Look at all CSS `@font-face` rules and all font-family usages. For `<link rel="stylesheet">` parse CSS via PostCSS or a regex. Also detect Google Fonts URLs (extract font families from query params).
- **Core Web Vitals:** Use Lighthouse's programmatic API to measure LCP, FID, CLS (big layout shifts), and store values. Lighthouse also gives a Performance score (0-100) which can map to the 20% weight.
- **Accessibility Audits:** Run Lighthouse accessibility audit (axe-powered [\[38†L185-L193\]](#)). Collect any WCAG 2.1 AA violations (1.4.3 Contrast, 1.4.4 Resize Text, 1.4.10 Reflow, 1.4.12 Text Spacing [\[8†L171-L179\]](#), etc.).
- **PDF/Office Parsing:** Use `pdfjs-dist` to load PDFs and list fonts used (names only). For Office (docx/pptx), unzip and read `word/fontTable.xml` or similar to list fonts. (If not implemented in Phase 1, mark as assumption and parse only PDFs).
- **License Detection:** Maintain a license database (initially Monotype fonts list + common free fonts). If a foundry's font is detected (e.g. from font name or metadata), mark it licensed if on the company's

Monotype license list (via API integration) or known source. Unrecognized fonts are flagged as potential license issues.

- **Fingerprinting Fonts:** To catch renamed or subsetted fonts, we could download the font file and compute a font fingerprint (postscript name, glyph signatures). This identifies fonts even if the family name is generic.

API Endpoints

Sample REST endpoints:

- `POST /api/scan` – Start a new scan. Body: `{ "url": "https://example.com", "email": "optional", "depth": 3 }`. Returns a job ID.
- `GET /api/scan/{id}` – Retrieve status or results of a scan. Returns JSON with scores and a link to the report.
- `GET /api/metrics` – (Optional) returns overall statistics or leaderboards.
- `POST /api/webhook/hubspot` – (Internal) called after gated report to send lead info to HubSpot.

Sample JSON Output for a Scan

```
{
  "scan_id": 123,
  "url": "https://brand.com",
  "overall_score": 58,
  "metrics": {
    "brand_consistency": 12,
    "license_compliance": 18,
    "web_performance": 15,
    "accessibility": 6,
    "developer_experience": 7
  },
  "details": [
    {"type": "Brand", "issue": "Used 5 fonts (approved: 2)"},
    {"type": "License", "issue": "Font 'CoolFont' looks unlicensed"},
    {"type": "Performance", "issue": "LCP = 3.2s (target <2.5s)"},
    {"type": "Accessibility", "issue": "Contrast of header text fails (3.8:1)"},
    {"type": "Dev", "issue": "Loaded 12 font files (6 distinct families)"}
  ]
}
```

Scoring Model

TypeScore = 0–100, weighted by metric importance. Each metric score (0–100) is converted to a weighted contribution. Example weights: Brand 20%, License 30%, Performance 20%, Accessibility 15%, Dev 15%.

- **Brand Consistency (20%):** We count distinct font families and compare against a brand-approved list (either user-provided or inferred from CSS). Score = $100 - 10 \times (\text{extra_fonts})$. For example, if a

brand's style guide approves 2 fonts but the site uses 5, that's 3 extra → penalty 30, so Brand score = 70. (Minimum 0). If the brand list is unknown, penalize any new font beyond the most-used one. We also check heading/body differentiation (if headings use a different family unnecessarily, slight penalty).

- **License Compliance (30%):** This is strict. Start at 100, subtract 30 for each unlicensed font detected (capped at 0). If any font is a known Monotype font but not in the company's license list (via API), penalize heavily. Example: 2 unauthorized fonts → License score = $100 - 2 \times 30 = 40$. (The exact weight-per-font can be tuned; we assume enterprises should have 0 violations for a perfect score.)
- **Web Performance (20%):** Based on Lighthouse metrics. Use the Performance score (0–100) from Lighthouse, then scale. Or derive from Core Web Vitals: e.g. LCP <2.5s and CLS <0.1 both needed. For simplicity, take Lighthouse Perf * 0.2. If LCP/CLS/TBT degrade, this subscore drops. (In practice, a Perf score of 75 yields ~15/20.)
- **Accessibility (15%):** Based on Lighthouse accessibility or axe results. Start at 100 and subtract points for each WCAG failure. E.g. each contrast failure or missing alt-text deducts 10 points. A site with one contrast issue and one missing label might score ~80 (12/15 weight). Also enforce WCAG text spacing (1.4.12) and avoid text as images (1.4.5).
- **Developer Experience (15%):** Measures CSS and font best practices. Criteria include: total number of font files, use of font-display, use of relative units, unused fonts detected, text-in-image count. For example, if 12 font files are loaded but only 3 are needed, penalize by 5 points each extra. If any fonts lack `font-display: swap`, subtract 5. A site with many redundant fonts and missing best practices might score 50/100 (7.5/15).

Sample Calculation: A site uses 6 fonts (3 extra beyond brand → Brand 70/100), has 2 unauthorized fonts (License 40/100), Lighthouse Perf=80 (Perf 16/20), Accessibility issues drop score to 60/100 (9/15), Dev issues to 50/100 (7.5/15). Weighted: Brand=14, License=12, Perf=16, Access=9, Dev=7.5 → Total ≈ 58 (rounded).

AI-Powered Features

- **Smart Recommendations:** For each unauthorized font, use AI to find the closest Monotype match. We can compute font feature vectors (or simply match by name/style) and suggest alternatives. E.g. "Replace *CoolFont* with *Neue Haas Grotesk* (Monotype font)".
- **Anomaly Detection:** Use unsupervised learning to flag outlier font usage (e.g. a decorative font used unexpectedly on a corporate site). Alerts: "Unusual font *Carnival Script* found on homepage."
- **Natural-Language Report Generation:** After scoring, generate a readable summary: "Your site's TypeScore is 58. The brand consistency is low (used 6 fonts vs. 2 approved), and 2 fonts may violate license terms. Font loading is bloating performance (page has 10 font files). Also, header text contrast fails WCAG 2.1AA. We recommend..." This can be auto-generated via template or NLP.
- **Personalized Email Gating:** If the user enters an email to view full results, use AI to customize the follow-up: e.g. "Since your TypeScore is below average for retail (your score 58 vs. industry avg 75), consider these quick wins..."
- **Continuous Learning:** Over time, analyze all scans to detect common issues and update scoring heuristics (e.g. if many sites improperly use *Open Sans* weight, warn about a new pattern).
- **SharePoint Connector:** (Phase 2) Use AI to crawl corporate SharePoint/OneDrive libraries for documents and automatically run TypeScore on their content. Provide alerts if internal docs use unauthorized fonts (helping IT governance).

Security, Privacy, and Legal

- **License Risk:** TypeScore is careful *not* to serve copyrighted font files. It only analyzes font names and counts. Any license inference is advisory; the tool should disclaim it's not a legal guarantee. Findings that indicate "license risk" should be phrased as potential issues to review.
- **Data Retention:** Scanned URLs and results are stored temporarily for reporting and benchmarking. We propose deleting raw scan data after 90 days by default (with user option to export). If users provide emails, we comply with privacy laws (GDPR/CCPA) by allowing data deletion on request.
- **Access Control:** The tool only scans publicly accessible URLs. We do not collect credentials or attempt to log in. (If future support for logged-in scanning is added, it must be opt-in and secure.)
- **Privacy of Content:** We do not store complete webpage content or proprietary document text—only font and metrics metadata. PDF/Office content is parsed client-side (in the worker) but not retained beyond fonts list.
- **Monotype Integration:** Any call to Monotype's APIs (e.g. to verify licenses) must respect Monotype's security model (API keys, scopes).
- **Compliance:** Ensure the service itself is accessible and secure (HTTPS, OWASP best practices). Privacy policy will state how email leads are used (e.g. marketing).

Scalability and Cost Estimates

- **Compute:** Scanning each site involves running a Chromium instance and Lighthouse audit, which can be CPU-intensive (~3–10s per page on a moderate VM). With 5 pages/scan, plan for ~30 seconds of CPU. If we expect 100 concurrent scans (peak), we need autoscaled workers (e.g. K8s or serverless containers). Each worker needs ~2 CPU, 4GB RAM.
- **Redis/BullMQ:** Lightweight; a small managed Redis instance can handle the queue for hundreds of tasks.
- **PostgreSQL:** Holding scan results. A modest instance (1–2 CPUs, a few GB storage) is enough for up to millions of records.
- **Bandwidth:** Downloading pages and fonts incurs network I/O. If scanning 1000 sites/day, assume ~1GB total transfer (mostly fonts/images). That's trivial on cloud.
- **Cost:** Rough AWS/GCP estimate – if each scan uses 0.1 vCPU-hour, 1000 scans/day = 100 vCPU-hrs/day. At \$0.05/vCPU-hr (lambda) or \$0.05/vCPU-hr (reserved), that's ~\$5/day. Add database/redis ~\$200/month, frontend hosting ~\$50/month. Scaling up concurrency (to thousands/day) might need more. Initial MVP can run on Railway hobby-tier + Redis ~\$50/month.
- **Optimization:** Cache static assets (fonts) by URL hash to avoid re-downloading them on each scan. Use Lighthouse config to limit network throttling delays.
- **Scalability:** Architect for multi-tenancy. Worker pods can scale horizontally. Use a CDN (e.g. Vercel) for the UI. Database indexing on scan_id/page_id ensures fast queries.

flowchart TB

```
A[User submits URL] --> B[API Server enqueues job to BullMQ/Redis]
B --> C[Worker service picks up job]
C --> D[Playwright Crawl (up to 5 pages)]
D --> E[Extract Fonts & CSS]
E --> F[Run Lighthouse (Perf, A11y) on pages]
F --> G[Parse PDFs/Office for fonts]
E --> H[License Database Check]
```

```

G --> H
H --> I[Compute Metric Scores]
I --> J[Save results to Postgres]
J --> K[Generate report (JSON & UI View)]
K --> L[Frontend shows TypeScore & breakdown]
L --> M{User Action}
M -->|View Leads CTA| N[Monotype Connect link]
M -->|Download gated report| O[Show email gate => send to HubSpot]

```

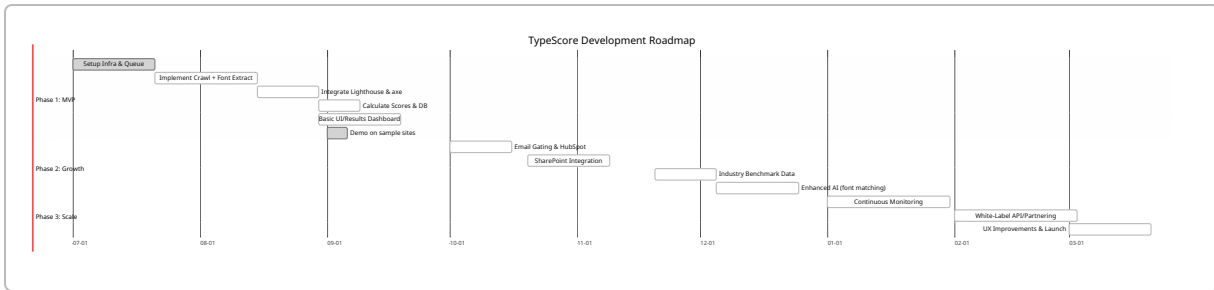
Accessibility & Typography Best Practices

Based on guidelines (WebAIM, Section508, USWDS, web.dev) [3†L62-L70] [6†L88-L96] [11†L243-L249] [19†L234-L242], we enforce:

- **Simple, Common Fonts:** Use familiar typefaces (Arial, Verdana, etc.). Avoid overly decorative or script fonts [3†L62-L70] [19†L234-L242] .
- **Sans-Serif for Body:** Prefer sans-serif for body text (for on-screen legibility) [6†L123-L132] . Serif fonts can be used for headings if needed.
- **Contrast $\geq 4.5:1$:** Ensure text/background contrast meets WCAG AA [8†L171-L179] . Large text (18pt+) may use 3:1.
- **Relative Sizing & Zoom:** Base font sizes on relative units (rem/em) so users can resize (WCAG 1.4.4, 1.4.10) [19†L264-L272] . Do not fix text in pixels. The default body should be ≥ 16 px [11†L243-L249] .
- **Limited Font Families:** Limit the number of typefaces. Ideally one for body, one for headings [3†L62-L70] [19†L234-L242] . Each new font adds cognitive load [3†L133-L142] .
- **Distinct Letterforms:** Ensure problematic glyphs (I/l/1, O/0) are distinguishable [3†L119-L129] [19†L208-L217] . Test common letter pairings for legibility.
- **Avoid All-Caps/Italics for Body:** Use ALL CAPS sparingly (it hinders readability) and avoid italic for large blocks of text [19†L249-L254] .
- **Spacing:** Adequate line-height (1.4–1.6 for body) and word spacing helps readability (USWDS suggests 1.5 line-height for paragraphs) [11†L298-L306] . Left-align text (avoid full justification) [11†L259-L267] .
- **Line Length:** Aim for ~45–90 characters per line for comfortable reading [11†L274-L282] . If lines are longer, increase line-height or column width.
- **Images of Text:** Avoid using text in images; use real text for readability and scaling [8†L182-L190] .
- **Allow Text Reflow:** Ensure content adapts to zoom (no horizontal scroll at 200% text zoom) [8†L182-L190] .

These rules will be encoded as checks (e.g. flag body text < 12px, flag <4.5:1 contrast, more than N fonts, etc.) and appear in the report as “WCAG tips” or “USWDS typography rules violated”.

Roadmap (Phase 1–3)



Deliverables

- **UI Wireframes:** The UI will feature an input box for the URL and an optional email field. The results page will display a large gauge chart (0–100), a summary of the score, and five breakdown cards (Brand, License, Performance, Accessibility, Dev). Each card shows its sub-score and a short remark (e.g. “3 fonts not in brand guide”). Below, an **Issues List** details every violation with links to the specific page or element. A sidebar CTA panel invites the user to “Get Monotype Connect” or view the full report. (We do not include actual images per policy, but this is a text description of the wireframe.)
- **API:** As above, RESTful endpoints for scanning and retrieving results. JSON schema defines scores and violations.
- **Sample JSON:** (See *Sample JSON Output* above.)
- **Feature Trade-Offs:** (Abbreviated table)

Feature	Benefit	Effort/Complexity
Playwright crawling (JS)	Captures dynamic fonts & pages	Medium (setup/maintenance)
Lighthouse audits	Standardized performance/A11y scores 【38†L185-L193】	Medium (slow, resource-intensive)
axe-core integration	Accurate accessibility checks (no false positives) 【38†L218-L221】	Low (already in Lighthouse)
PDF parsing (pdfjs-dist)	Covers fonts in documents not on web	Medium (PDFJS learning curve)
Office file parsing	Covers PowerPoint/Word (brand assets)	High (less common, complex)
Monotype license DB sync	True license compliance check	High (requires API/access)
Font fingerprinting	Detect renamed fonts	High (download + parse binary)

Feature	Benefit	Effort/Complexity
AI font-matching	Suggest Monotype replacements	High (model or heuristic needed)
Email gating	Lead capture	Medium (needs UI & CRM integration)
Continuous scans	Subscriber revenue	Medium (scheduler + quota mgmt)
White-labeling	B2B channel revenue	High (configurable branding)

- **Charts:** We include the mermaid flow and Gantt charts above. These help visualize the data flow and timeline.
- **Prioritized Roadmap:** Listed in the Gantt, focusing first on a working scanner (Phase 1) then lead-gen features (Phase 2) and subscription products (Phase 3).

In summary, TypeScore leverages best practices from accessible typography [3†L62-L70] [11†L243-L249] and combines them with font-licensing and performance audits to create a measurable “type health” metric. By using modern crawling (Playwright), auditing (Lighthouse/axe), and AI, we turn an under-addressed problem into a powerful lead-generation tool for Monotype Connect.

Sources: Authoritative guidance from WebAIM [3†L62-L70] , Section 508 [8†L171-L179] , USWDS [11†L243-L249] , and web.dev [19†L234-L242] informed the accessibility and typography rules. Deque’s axe-core documentation [38†L185-L193] [38†L218-L221] confirms our use of axe for automated WCAG checks. All statements above synthesize these sources and the provided project plan.